

LAPORAN PRAKTIKUM
SISTEM OPERASI

MODUL

6



Oleh:

(Rifqi M Ramdani)

(2311104044)

PROGRAM STUDI S1 REKAYASA LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2025

BAB I

PENDAHULUAN

A. Dasar Teori

Sistem operasi merupakan perangkat lunak yang berfungsi untuk mengelola sumber daya komputer dan menyediakan layanan bagi program aplikasi. Salah satu konsep penting dalam sistem operasi adalah proses, yaitu program yang sedang dieksekusi. Dalam sistem operasi modern, proses dapat berjalan secara sekuensial maupun konkuren. Proses sekuensial adalah proses yang dijalankan satu per satu secara berurutan, sedangkan proses konkuren adalah beberapa proses yang berjalan secara bersamaan dalam satu waktu (interleaving) yang dikelola oleh scheduler.

Pada sistem Xinu, setiap proses direpresentasikan sebagai struktur proses yang dapat dibuat menggunakan fungsi `create()` dan dijalankan menggunakan `resume()`. Xinu mendukung manajemen proses sederhana yang memungkinkan percobaan konsep konkurensi dan sinkronisasi.

Namun, dalam proses konkuren yang menggunakan variabel global bersama tanpa mekanisme sinkronisasi, dapat terjadi race condition, yaitu kondisi ketika beberapa proses mengakses dan memodifikasi data secara bersamaan sehingga menghasilkan output yang tidak konsisten.

B. Tujuan

Memahami konsep proses sekuensial dalam sistem operasi.

Memahami konsep proses konkuren pada sistem operasi Xinu.

Mempelajari cara pembuatan dan eksekusi proses menggunakan `create()` dan `resume()`.

Mengamati perbedaan hasil eksekusi proses sekuensial dan konkuren.

Menganalisis terjadinya race condition pada proses yang menggunakan variabel global bersama.

BAB II je.

HASIL

A. UNGUIDED (Tugas Mandiri)

1.

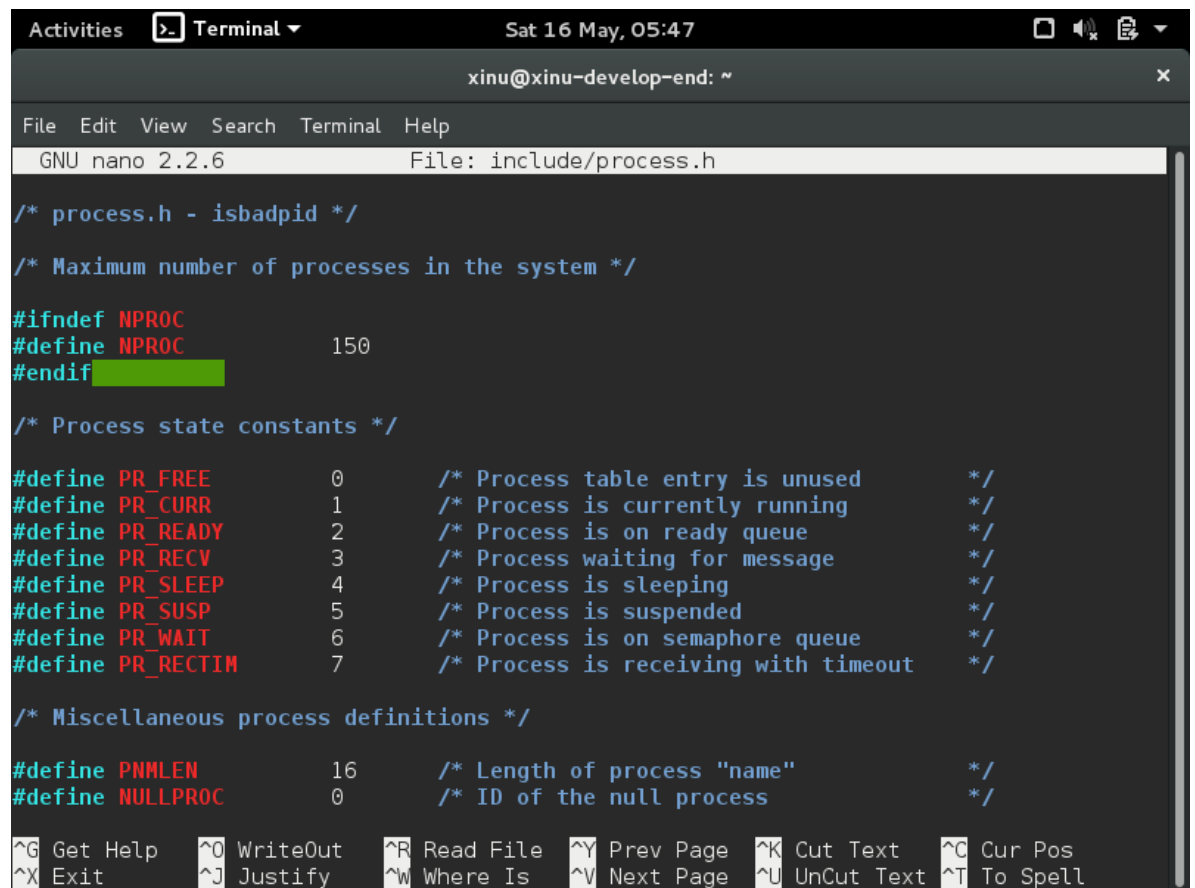
[10 Poin] Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan

secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767

proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang

bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!



```
Activities Terminal Sat 16 May, 05:47
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: include/process.h

/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifndef NPROC
#define NPROC 150
#endif

/* Process state constants */

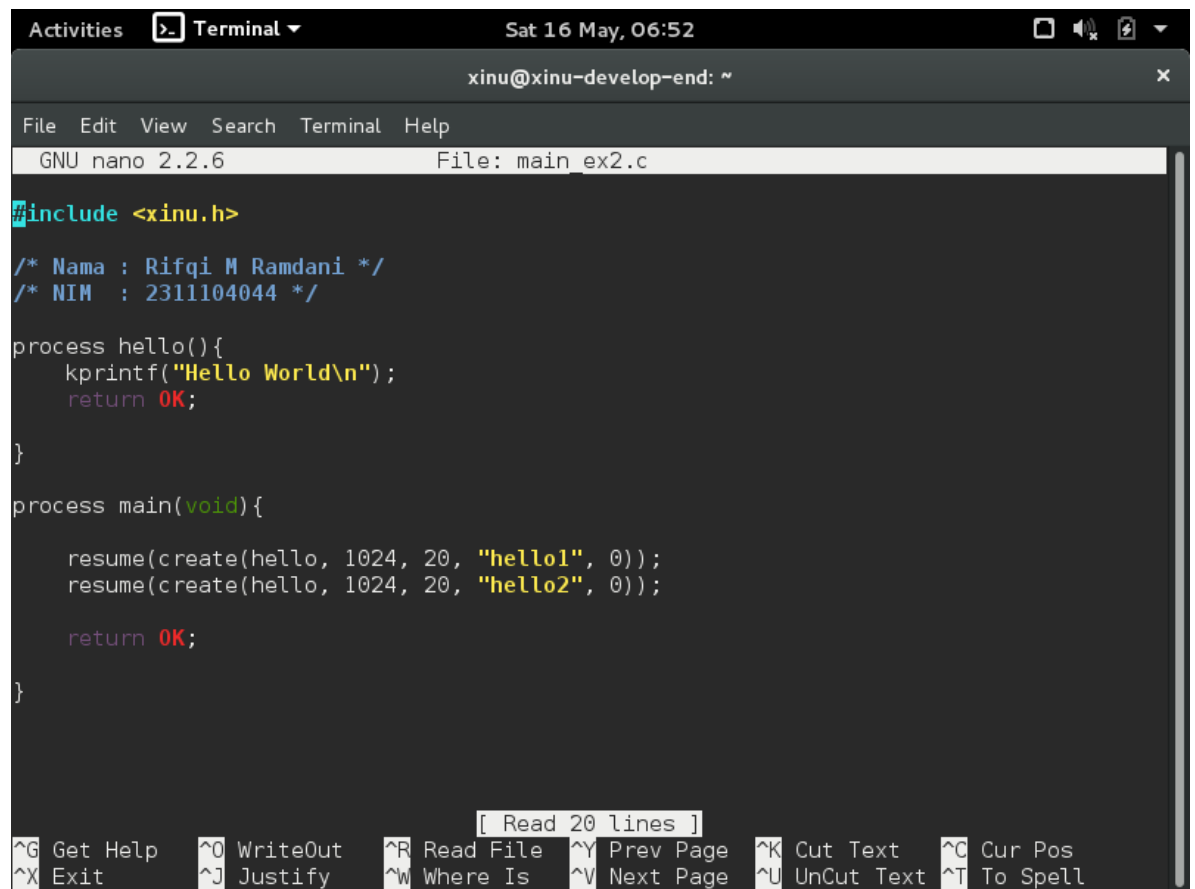
#define PR_FREE 0 /* Process table entry is unused */
#define PR_CURR 1 /* Process is currently running */
#define PR_READY 2 /* Process is on ready queue */
#define PR_RECV 3 /* Process waiting for message */
#define PR_SLEEP 4 /* Process is sleeping */
#define PR_SUSP 5 /* Process is suspended */
#define PR_WAIT 6 /* Process is on semaphore queue */
#define PR_RECTIM 7 /* Process is receiving with timeout */

/* Miscellaneous process definitions */

#define PNMLEN 16 /* Length of process "name" */
#define NULLPROC 0 /* ID of the null process */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

2. [20 Poin] Jalankan kode main_ex1!



```
Activities Terminal Sat 16 May, 06:52
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: main_ex2.c

#include <xinu.h>

/* Nama : Rifqi M Ramdani */
/* NIM : 2311104044 */

process hello(){
    kprintf("Hello World\n");
    return OK;
}

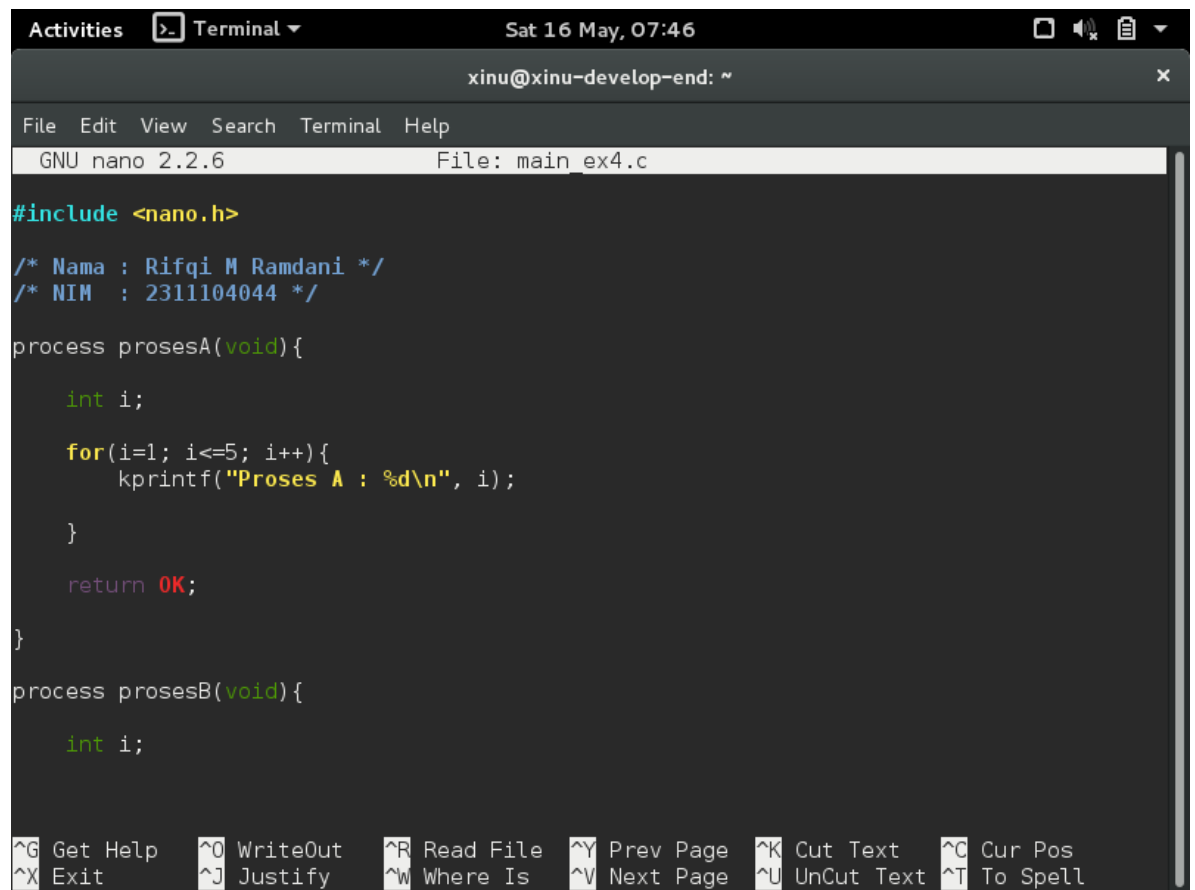
process main(void){

    resume(create(hello, 1024, 20, "hello1", 0));
    resume(create(hello, 1024, 20, "hello2", 0));

    return OK;
}

[ Read 20 lines ]
^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

3. [20 Poin] Jalankan kode main_ex2!



```
Activities Terminal Sat 16 May, 07:46
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: main_ex4.c

#include <stdio.h>

/* Nama : Rifqi M Ramdani */
/* NIM : 2311104044 */

process prosesA(void){
    int i;
    for(i=1; i<=5; i++){
        kprintf("Proses A : %d\n", i);
    }
    return OK;
}

process prosesB(void){
    int i;
```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

Activities Terminal Sat 16 May, 07:48

xinu@xinu-develop-end: ~

File Edit View Search Terminal Help

GNU nano 2.2.6 File: main_ex4.c

```
        kprintf("Proses B : %d\n", 1);
    }

    return OK;
}

process main(void){
    resume(create(prosesA, 1024, 20, "A", 0));
    resume(create(prosesB, 1024, 20, "B", 0));

    return OK;
}
```

Get Help	WriteOut	Read File	Prev Page	Cut Text	Cur Pos
Exit	Justify	Where Is	Next Page	UnCut Text	To Spell

Activities Terminal Sat 16 May, 07:48

xinu@xinu-develop-end: ~

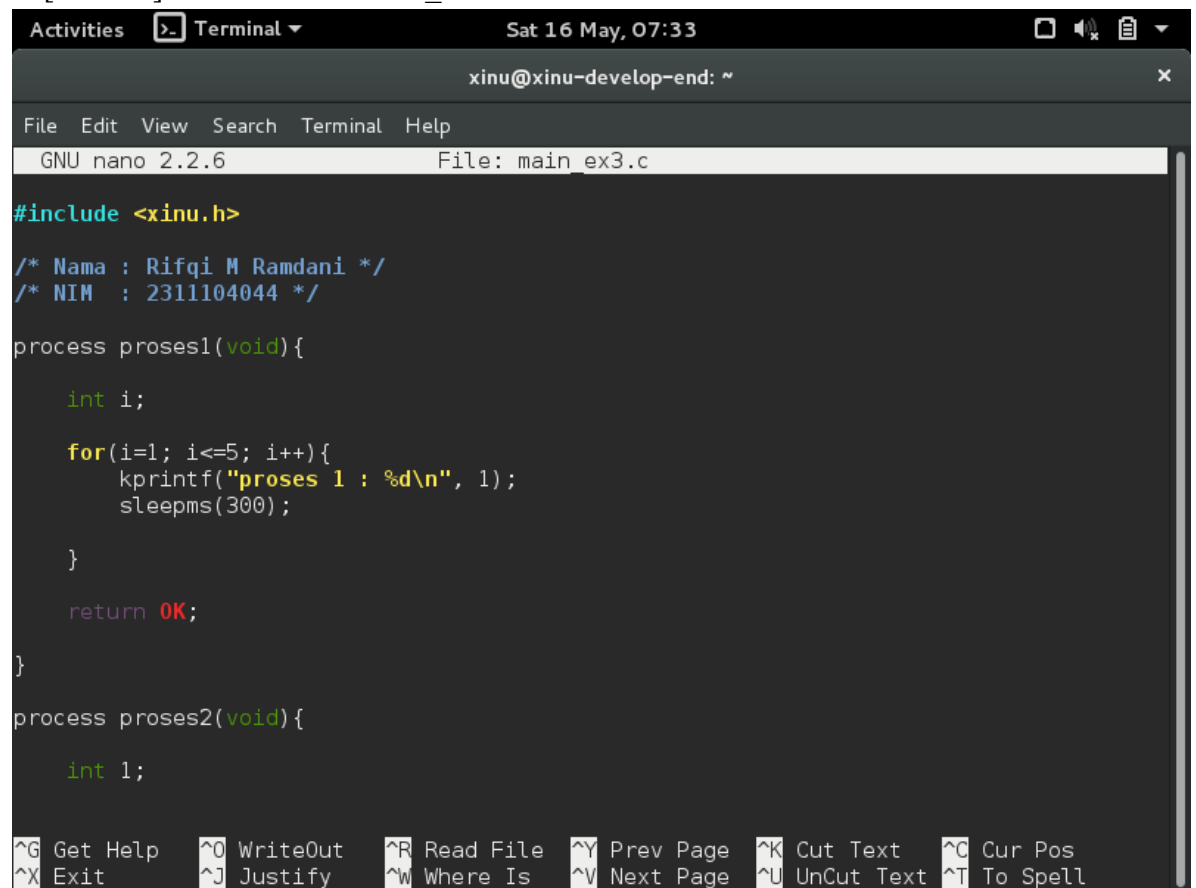
File Edit View Search Terminal Help

GNU nano 2.2.6 File: main_ex4.c

```
}  
    return OK;  
}  
process prosesB(void){  
    int i;  
  
    for(i=1; i<=5; i++){  
        kprintf("Proses B : %d\n", i);  
    }  
  
    return OK;  
}  
process main(void){
```

Get Help	WriteOut	Read File	Prev Page	Cut Text	Cur Pos
Exit	Justify	Where Is	Next Page	UnCut Text	To Spell

4. [20 Poin] Jalankan kode main_ex3!



```
GNU nano 2.2.6 File: main_ex3.c

#include <xinu.h>

/* Nama : Rifqi M Ramdani */
/* NIM  : 2311104044 */

process proses1(void){
    int i;

    for(i=1; i<=5; i++){
        kprintf("proses 1 : %d\n", 1);
        sleepms(300);
    }

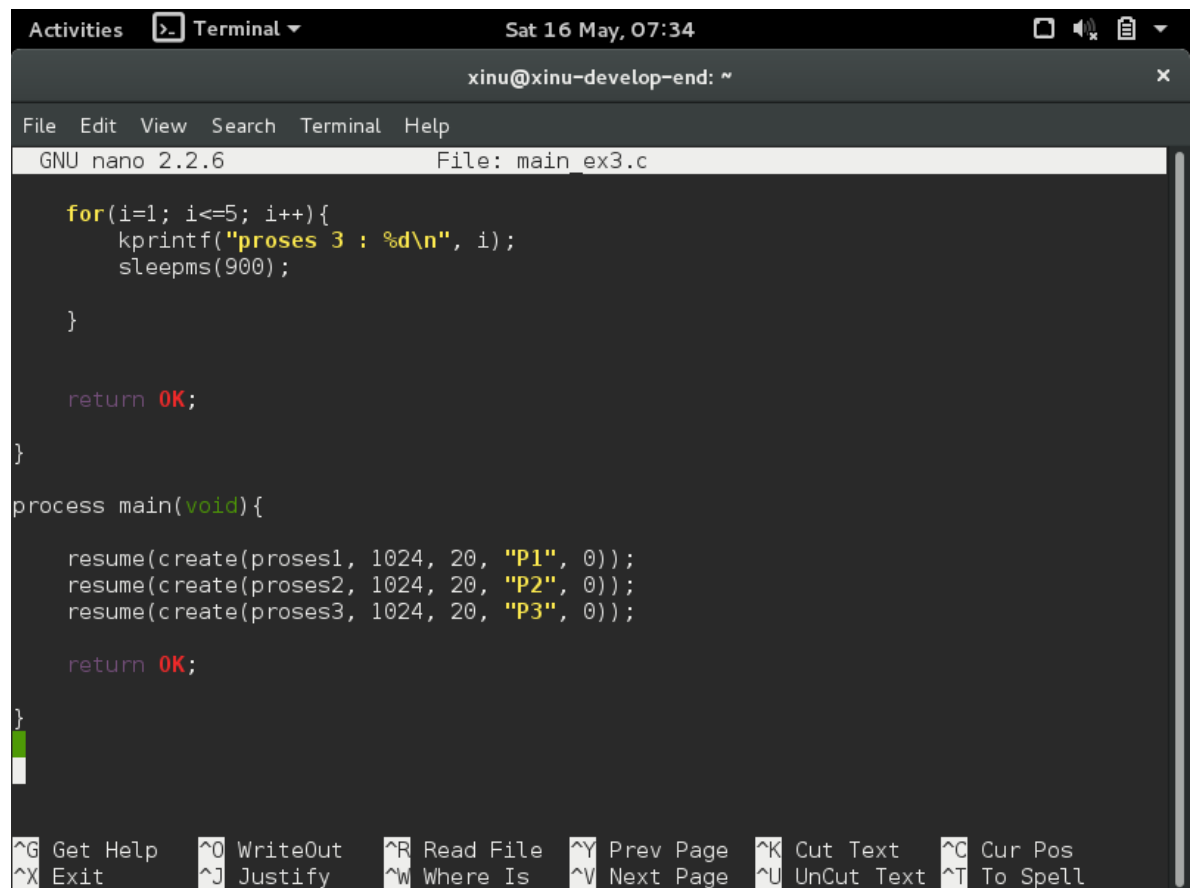
    return OK;
}

process proses2(void){
    int 1;
}
```

Terminal window title: xinu@xinu-develop-end: ~

Terminal window menu: File Edit View Search Terminal Help

Terminal window status bar: ^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos ^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell



```
GNU nano 2.2.6 File: main_ex3.c

    for(i=1; i<=5; i++){
        kprintf("proses 3 : %d\n", i);
        sleepms(900);
    }

    return OK;
}

process main(void){

    resume(create(proses1, 1024, 20, "P1", 0));
    resume(create(proses2, 1024, 20, "P2", 0));
    resume(create(proses3, 1024, 20, "P3", 0));

    return OK;
}
```

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

5. [30 Poin] Buatlah 2 proses produser dan konsumen. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```
Activities Terminal Sat 16 May, 09:21
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
-00 -DBREL0C=0x150000 -DB00TPL0C=0x150000 -DBSDURG -DVERSION=\"\"`cat version`\" -I../i
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBREL0C=0x150000 -DB00TPL0C=0x150000 -DBSDURG -DVERSION=\"\"`cat version`\" -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBREL0C=0x150000 -DB00TPL0C=0x150000 -DBSDURG -DVERSION=\"\"`cat version`\" -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBREL0C=0x150000 -DB00TPL0C=0x150000 -DBSDURG -DVERSION=\"\"`cat version`\" -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBREL0C=0x150000 -DB00TPL0C=0x150000 -DBSDURG -DVERSION=\"\"`cat version`\" -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBREL0C=0x150000 -DB00TPL0C=0x150000 -DBSDURG -DVERSION=\"\"`cat version`\" -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

root@xinu-develop-end:/home/xinu/xinu/compile# ls
bin binaries ld.script main_ex3.c Makefile version xinu.elf xinu.map
root@xinu-develop-end:/home/xinu/xinu/compile# xinu
bash: xinu: command not found
root@xinu-develop-end:/home/xinu/xinu/compile# ./xinu.elf
```

BAB III

PENUTUP

A. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa sistem operasi Xinu mendukung eksekusi proses secara sekuensial maupun konkuren. Proses konkuren memungkinkan beberapa proses berjalan secara bersamaan, namun dapat menimbulkan masalah seperti race condition jika menggunakan variabel global tanpa sinkronisasi.

Pada program produser dan konsumen, hasil keluaran tidak selalu sesuai dengan urutan yang diharapkan karena adanya akses bersamaan terhadap variabel global. Hal ini menunjukkan pentingnya mekanisme sinkronisasi dalam sistem operasi untuk menjaga konsistensi data pada proses konkuren.